CAHIER DES CHARGES TECHNIQUE

Projet : Cartes grises

Langages : MySQL – Java

Méthodologie : Agile Kanban (Trello)

BTS SIO – 2025/2026

Sommaire

1. Contexte du projet
    1.1. Présentation du projet
    1.2. Date de rendu du projet
2. Besoins fonctionnels
3. Ressources nécessaires à la réalisation du projet
    3.1. Ressources matérielles
    3.2. Ressources logicielles
4. Gestion du projet
5. Gestion du projet
6. Conception du projet
    6.1. Le front-end
        6.1.1. Wireframes
        6.1.2. Maquettes
        6.1.3. Arborescences
    6.2. Le back-end
        6.2.1. Diagramme de cas d’utilisation
        6.2.2. Diagramme de classe
        6.2.3. Diagramme d’activités
        6.2.4. Modèles Conceptuel de Données (MCD)
        6.2.5. Modèle Logique de Données (MLD)
        6.2.6.Modèle Physique de Données (MPD)
7. Technologies utilisées
    7.1. Langages de développement Web
    7.2. Base de données
8. Sécurité
    8.1. Protection contre les injections SQL

1. Contexte du projet

1.1. Présentation du projet

La Direction des systèmes d'information de la préfecture a été sollicitée par le service des cartes grises afin de faire évoluer leur application métier actuelle, permettant la gestion des dossiers de cartes grises.

Votre équipe et vous-même avez pour mission d'analyser l'application existante, de produire les documents de conception nécessaires, puis de proposer et mettre en œuvre une solution répondant aux besoins d'évolution exprimés par le service.

1.2. Date de rendu du projet

Le projet doit être rendu au plus tard le 15/01/2025.

2. Besoins fonctionnels

Le site web devra avoir une partie accessible au public et une partie privée permettant de gérer les données.

Les données seront stockées dans une base de données relationnelle pour faciliter la gestion et la mise à jour des informations. Ces données peuvent être gérées directement via le site web à travers un espace administrateur.

3. Ressources nécessaires à la réalisation du projet

3.1. Ressources matérielles

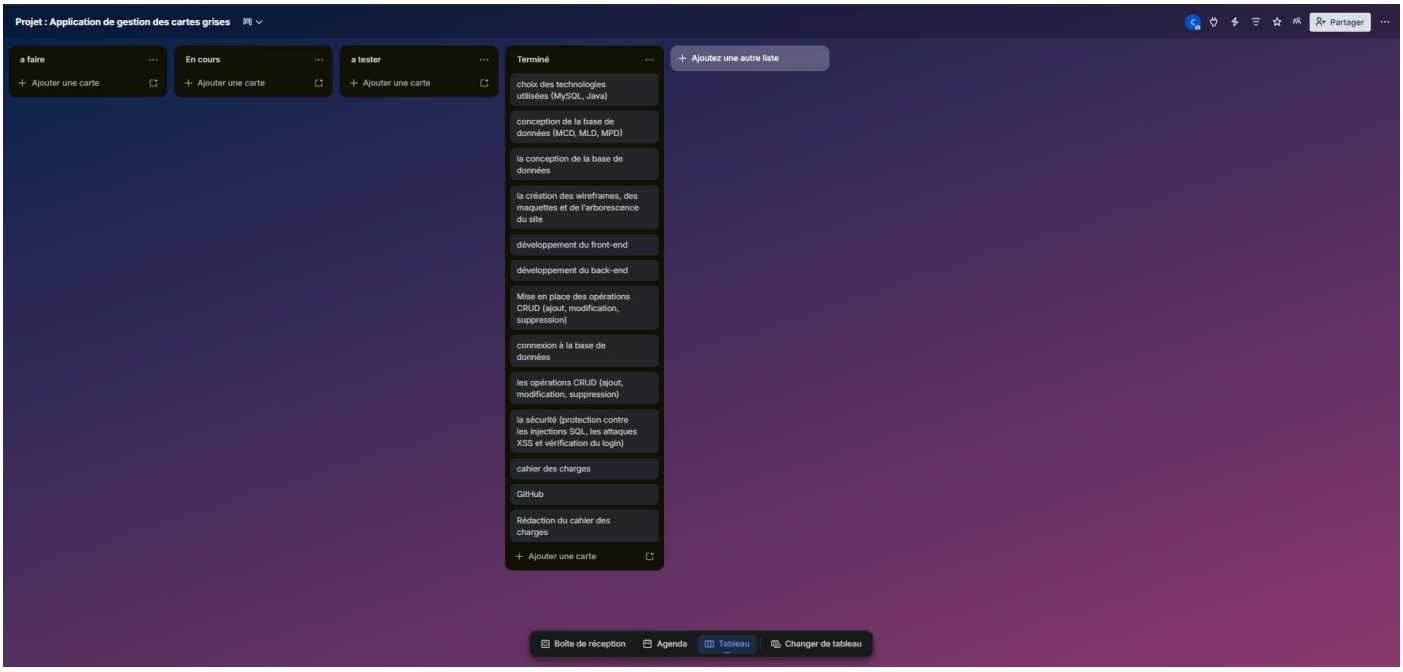
- PC
- Connexion internet
- Serveur local
- Périphérique de travail

3.2. Ressources logicielles

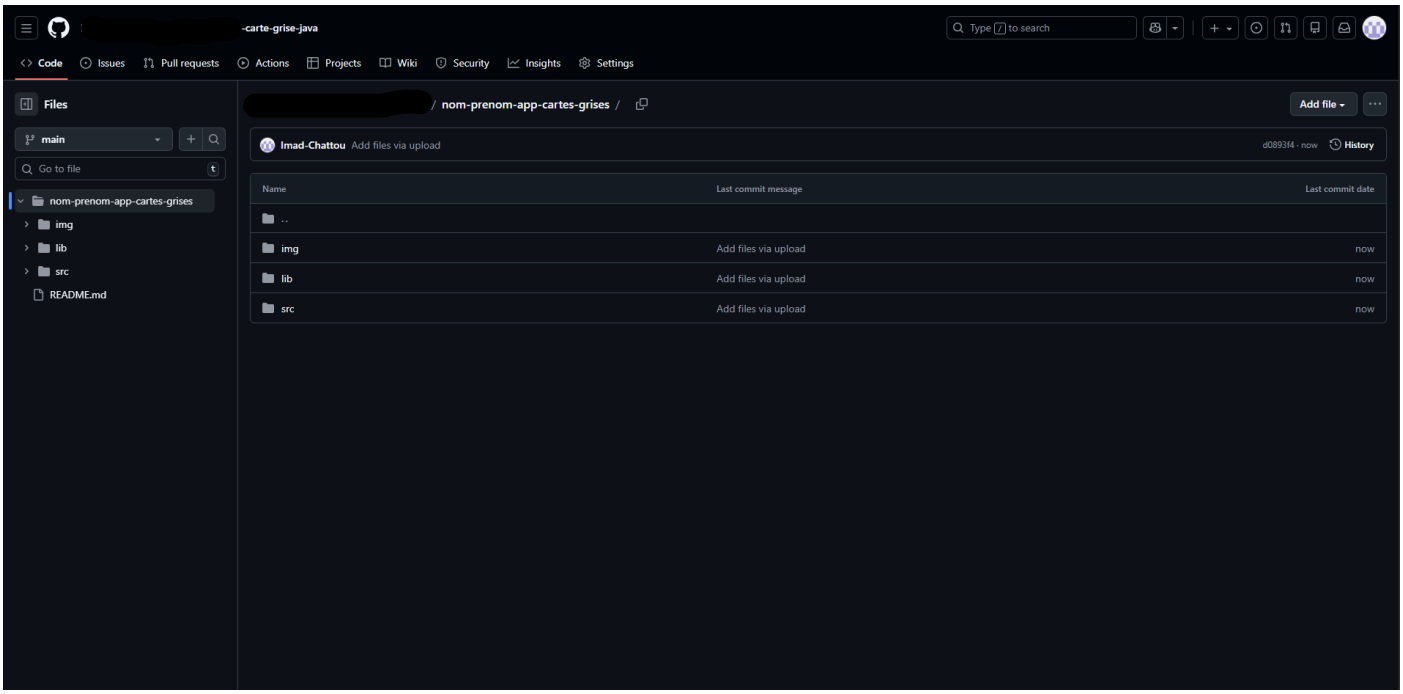
- Visual studio code
- Mysql
- Mamp
- Mocodo
- Java

4. Gestion du projet

Pour réaliser le projet, nous utiliserons la méthode Agile Kanban. Nous utiliserons également l'outil de gestion de projet en ligne Trello.



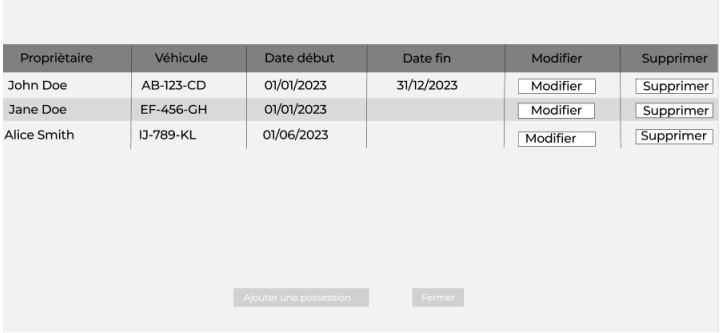
5. Partage collaboratif sur GitHub

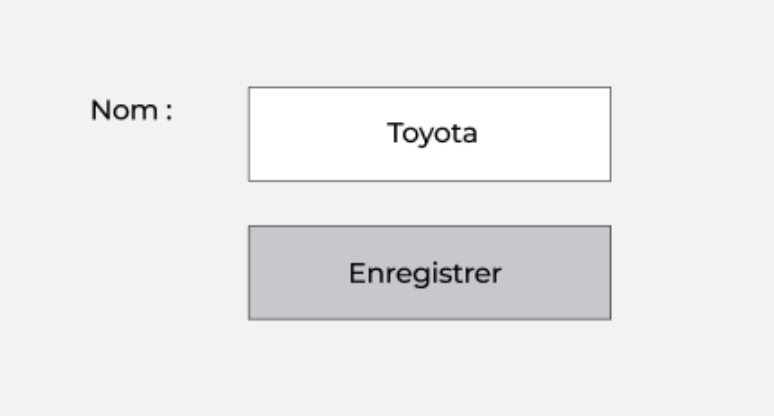


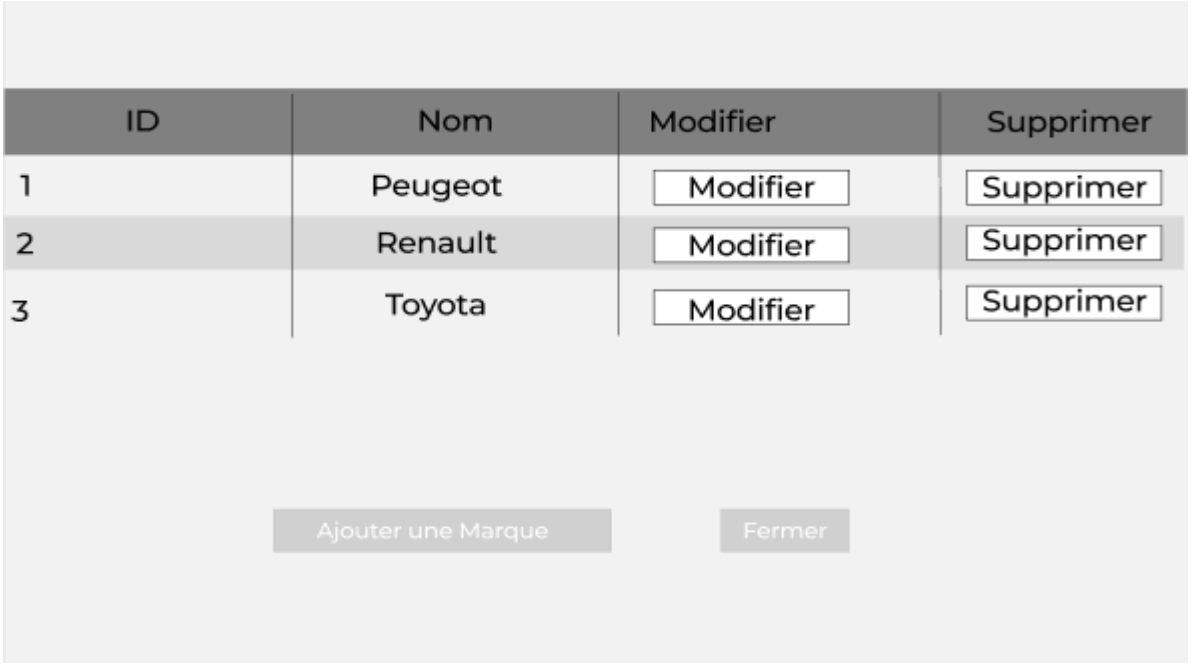
6. Conception du projet

6.1. Le front-end

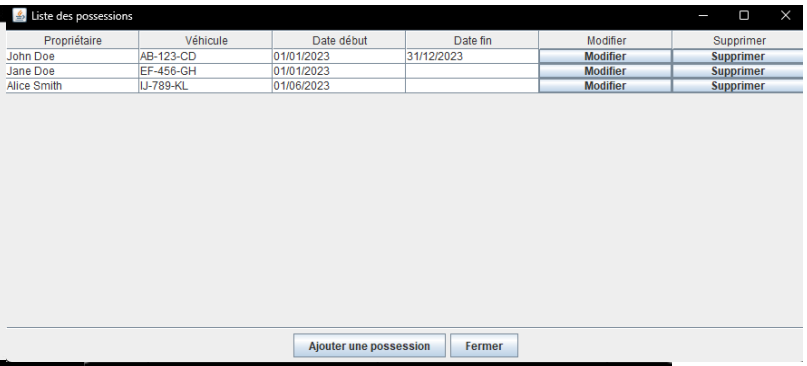
6.1.1. Wireframes

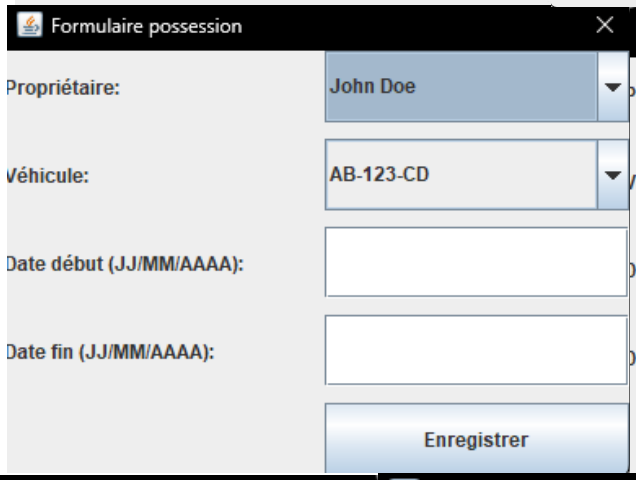


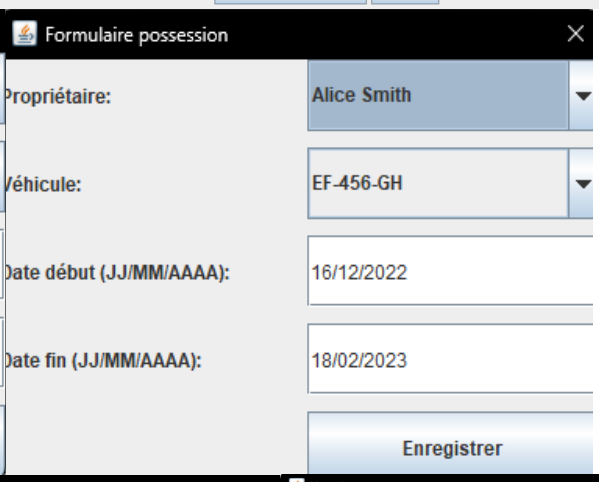


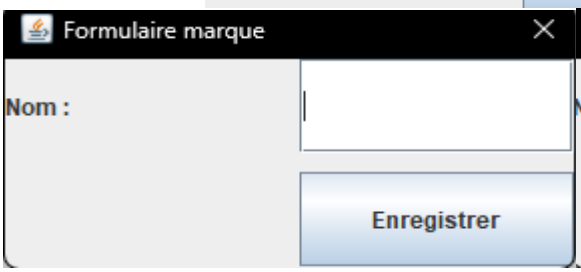


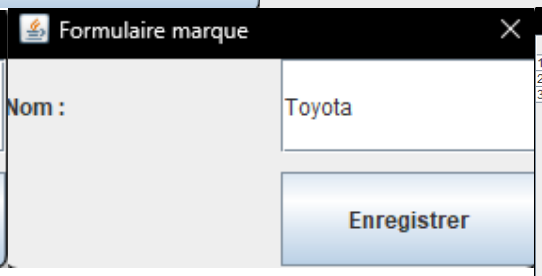
6.1.2. Maquettes



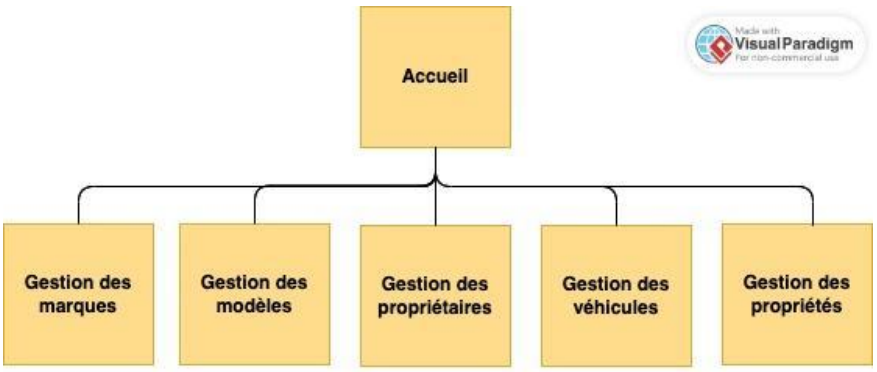






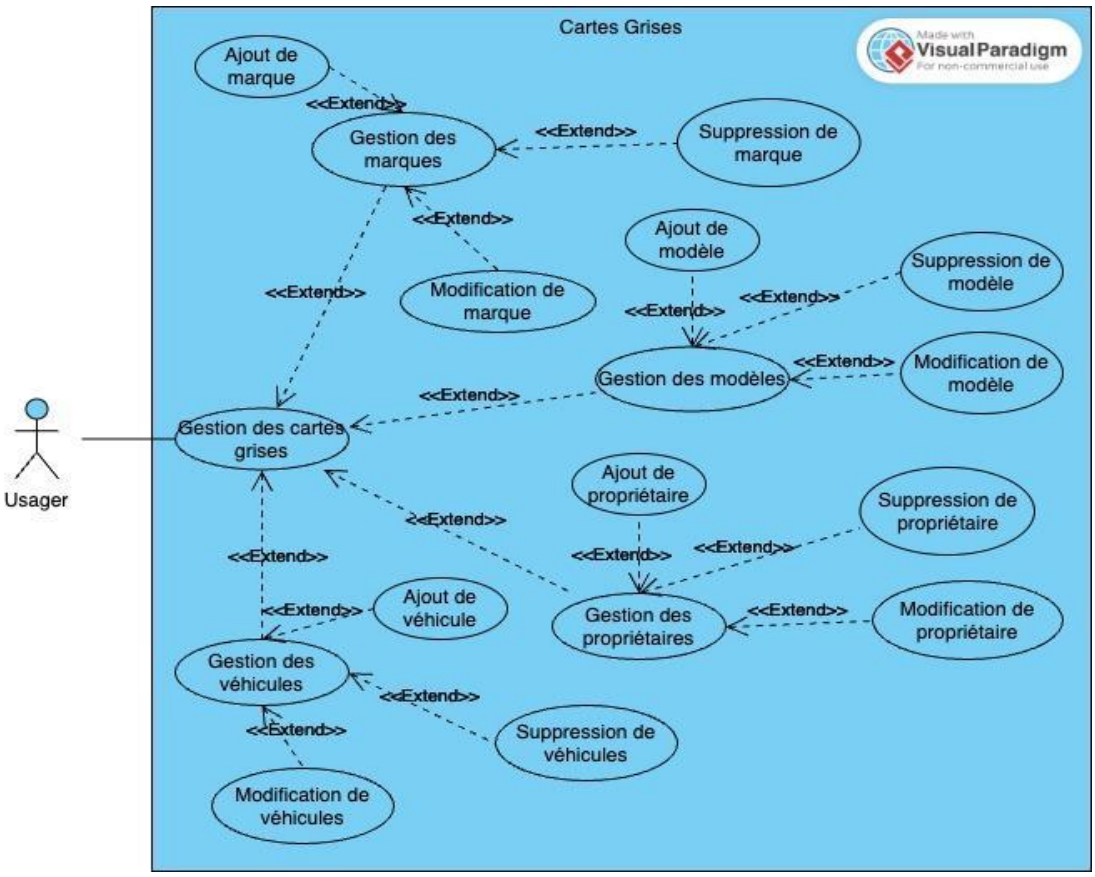


6.1.3. Arborescences

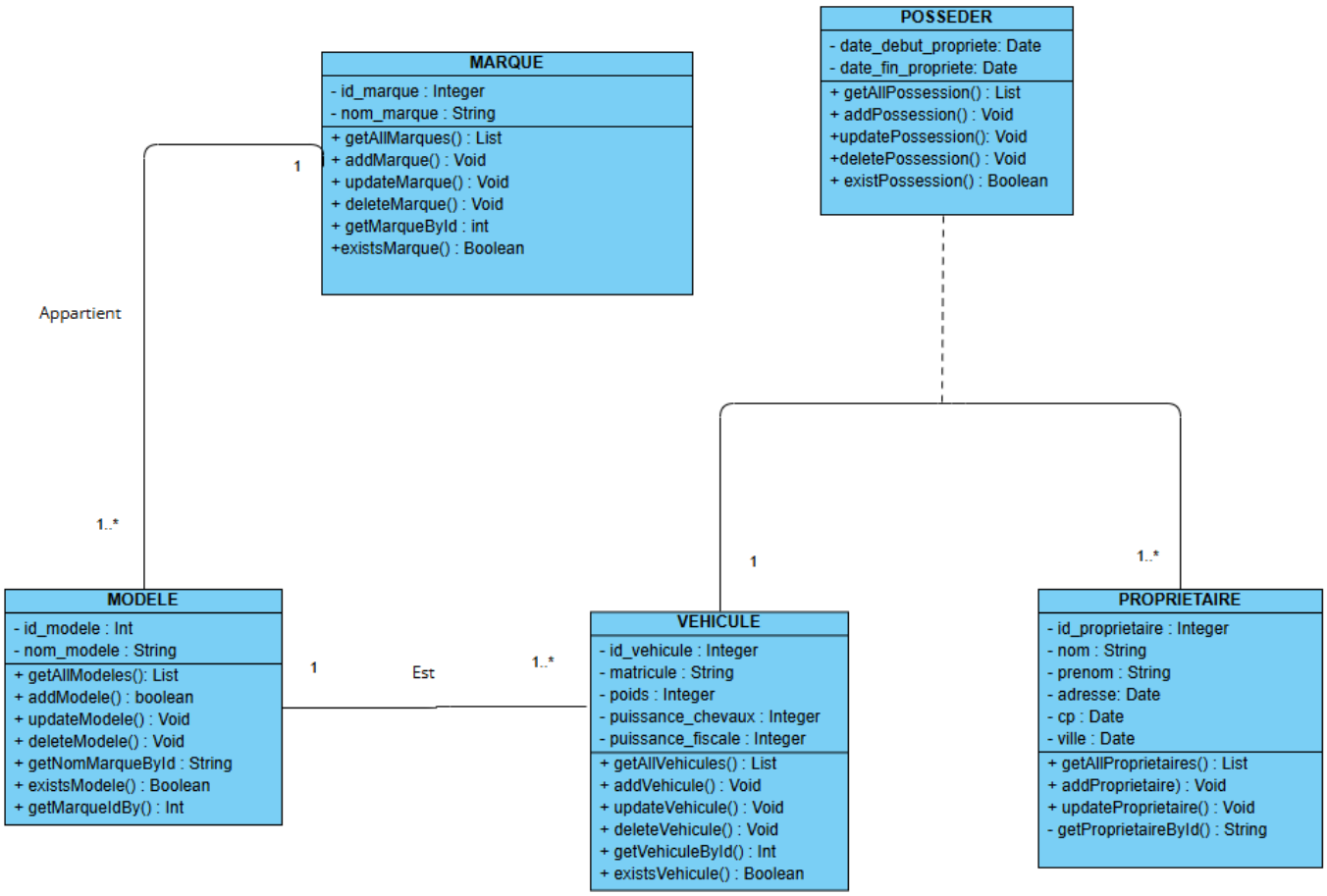


6.2. Le back-end

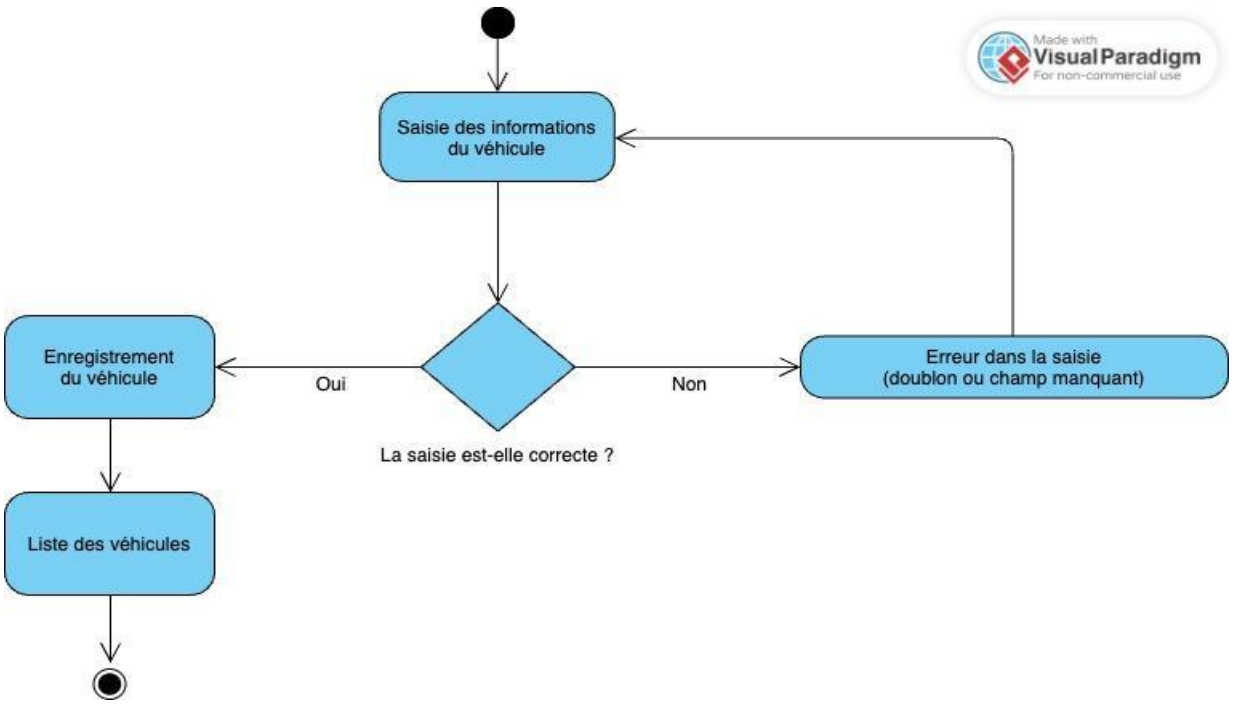
6.2.1. Diagramme de cas d'utilisation



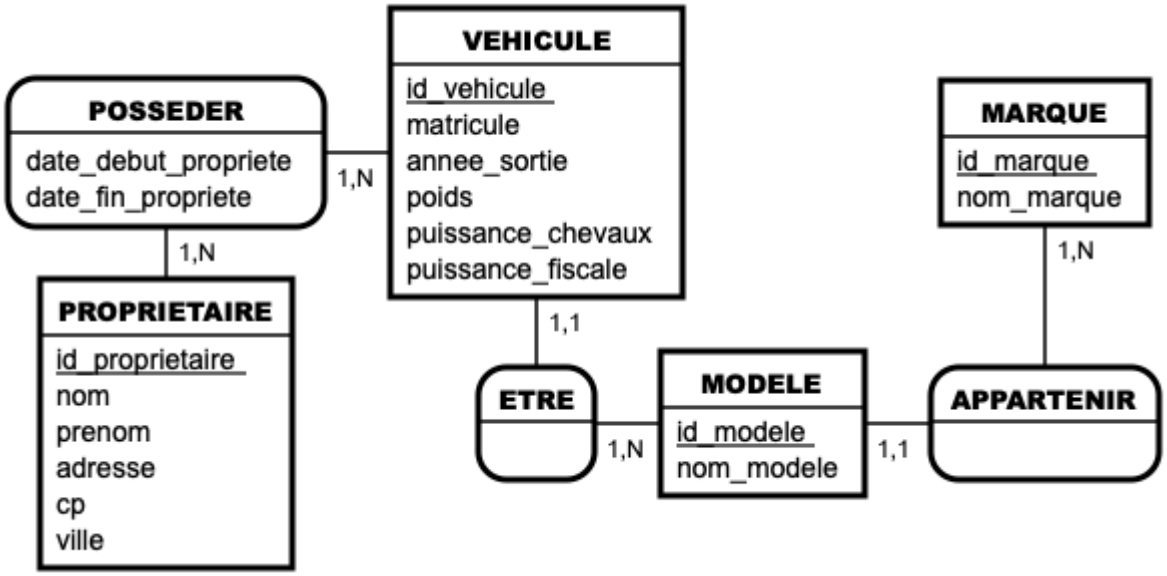
6.2.2. Diagramme de classe



6.2.3. Diagramme d'activités



6.2.4. Modèles Conceptuel de Données (MCD)



6.2.5. Modèle Logique de Données (MLD)

- **MARQUE (id_marque, nom_marque)**
  Clé primaire :id_marque

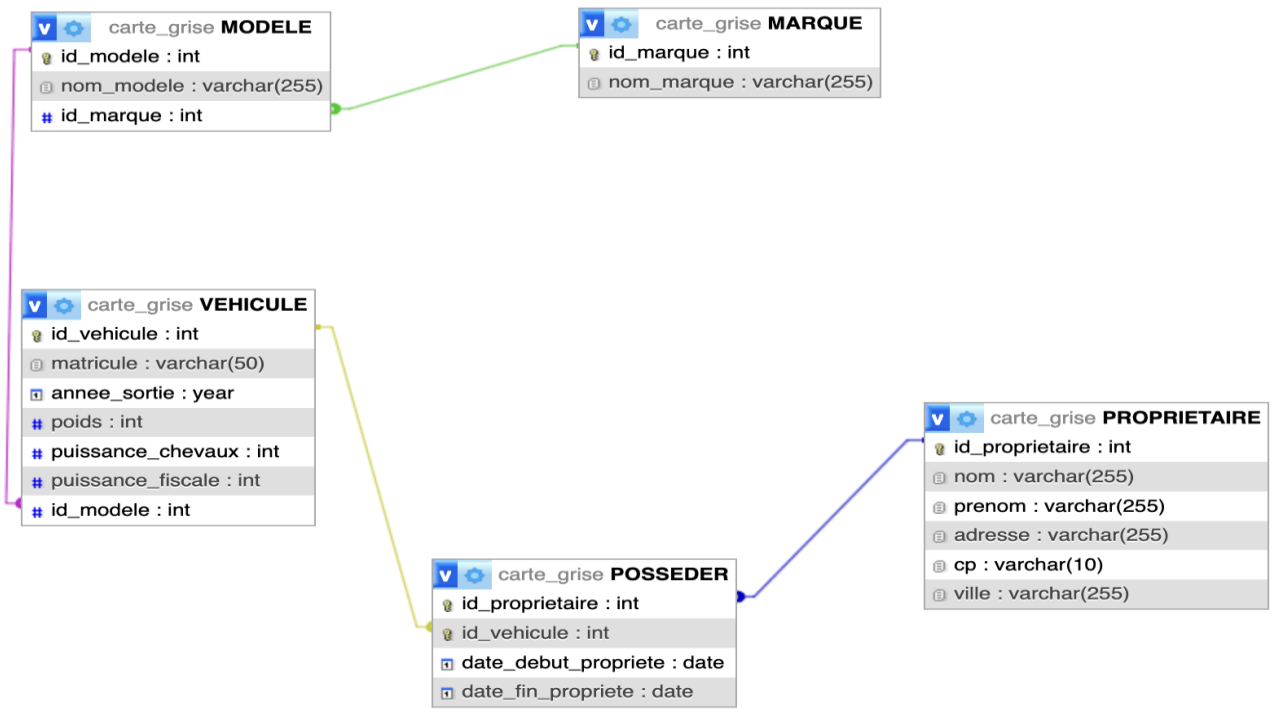
- **MODELE (id_modele, nom_modele, id_marque)**
  Clé primaire :id_modele
  Clé étrangère :id_marque en référence à id_marque de MARQUE

- **VEHICULE (id_vehicule, matricule, annee_sortie, poids, puissance_chevaux, puissance_fiscale, id_modele)**
  Clé primaire : id_vehicule
  Clé étrangère :id_modele en référence à id_modele de MODELE

- **PROPRIETAIRE (id_proprietaire, nom, prenom, adresse, cp, ville)**
  Clé primaire :id_proprietaire

- **POSSEDER (id_proprietaire, id_vehicule, date_debut_propriete, date_fin_propriete)**
  Clé primaire :id_proprietaire, id_vehicule
  Clés étrangères :id_proprietaire en référence à id_proprietaire de PROPRIETAIRE id_vehicule en référence à id_vehicule de VEHICULE

6.2.6. Modèle Physique de Données (MPD)



7. Technologies utilisées

7.1. Langages de développement Web

7.2. Base de données

8. Sécurité

8.1. Protection contre les injections SQL

Définition de la solution :

Injection SQL : Attaque de sécurité qui permet à un utilisateur malveillant d'interférer avec les requêtes SQL qu'une application exécute sur sa base de données.
Comment ça marche ?
L'attaquant insère du code SQL malveillant dans des champs de saisie (formulaires, URL, etc.) qui est ensuite exécuté par la base de données lorsqu'il est concaténé sans protection dans une requête SQL.
Conséquences possibles :

- Accès non autorisé aux données
- Modification/suppression de données
- Élévation de privilèges
- Exécution de commandes système

Extraits de code

Extrait : CRUD sécurisé avec protection SQL (Proprietaire / modèle)

```
// Ligne public static boolean addProprietaire(String nom, String prenom,
String adresse, String cp, String ville) {
    String sql = "INSERT INTO PROPRIETAIRE (nom, prenom, adresse, cp, ville)
VALUES (?, ?, ?, ?, ?)";

    try (Connection conn = DBConnection.getConnection();
         PreparedStatement ps = conn.prepareStatement(sql)) {

        ps.setString(1, nom);
        ps.setString(2, prenom);
        ps.setString(3, adresse);
        ps.setString(4, cp);
        ps.setString(5, ville);

        return ps.executeUpdate() > 0;

    } catch (SQLException e) {
        System.err.println("Erreur addProprietaire : " + e.getMessage());
        return false;
    }
}
```

Extrait : Interface graphique et interaction avec le contrôleur (ProprietaireView) :

```
JButton addButton = new JButton("Ajouter un propriétaire");
addButton.addActionListener(e -> showProprietaireForm(null));

JButton closeButton = new JButton("Fermer");
closeButton.addActionListener(e -> dispose()); // ferme uniquement cette
fenêtre
```

TECHNIQUES UTILISÉES DANS VOTRE CODE :

- Illustre l'utilisation de PreparedStatement pour éviter les injections SQL.
- Montre comment le modèle gère l'ajout de données dans la base de données de façon sécurisée.
- Montre comment la vue Swing interagit avec le contrôleur pour ajouter un nouveau propriétaire.
- Illustre la séparation MVC : la vue ne fait que déclencher les actions, le contrôleur gère la logique métier.